Distributed Trust

and

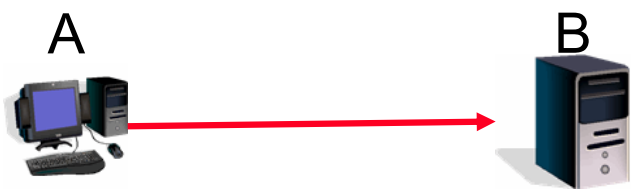
Byzantine Fault Tolerance

MSI – MEI – MI – MDS
2024/2025

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

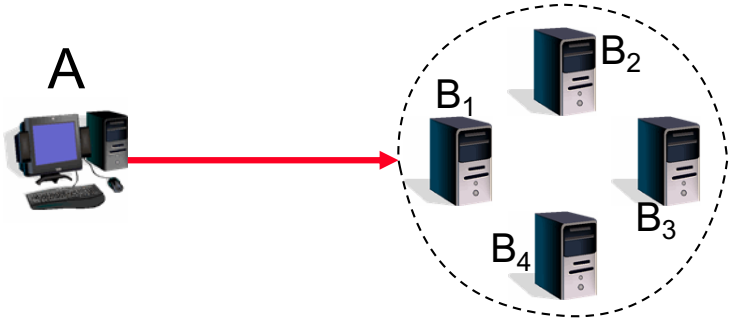
Trust and Distributed Trust

- **Trust: the accepted dependence of a component, on a set of properties of another component, subsystem or system**
- **Trustworthiness: the measure in which a component, subsystem or system meets a set of properties**



A trusts that B is correct

- **Distributed Trust: the accepted dependence of a component that is made trustworthy through means of distribution and removal of single point of failure**



A trusts that *most* B_i*s* are correct

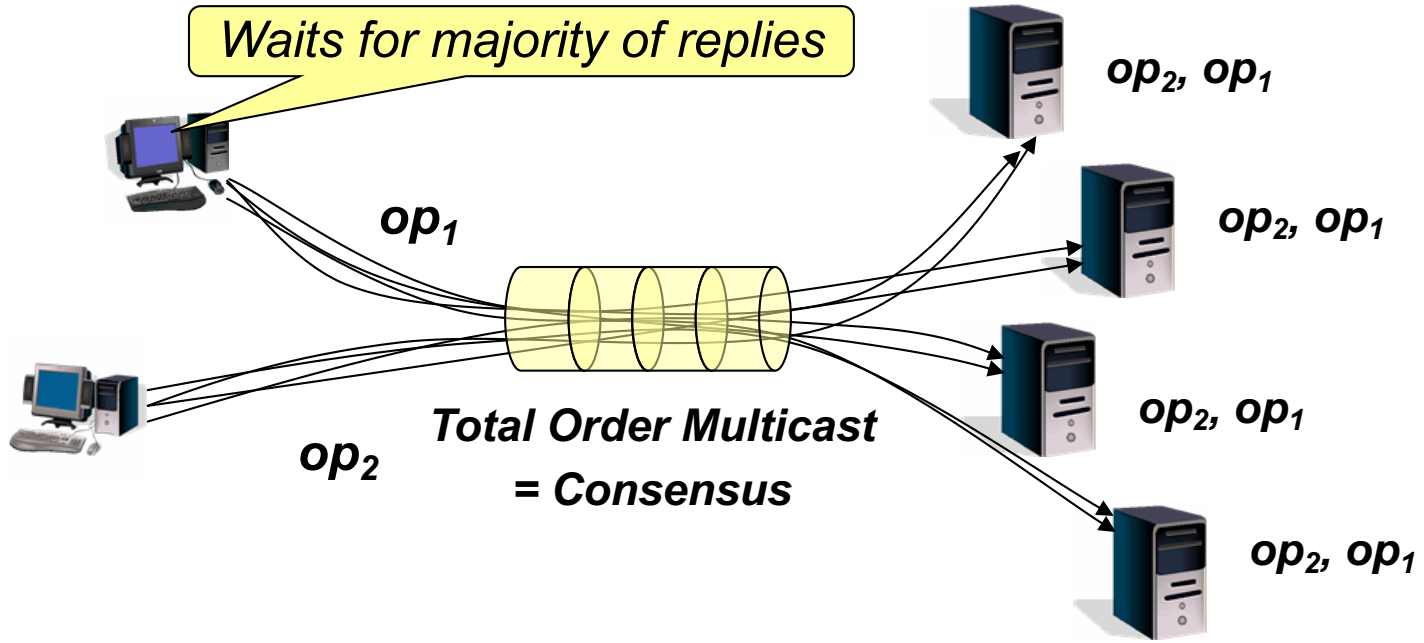
Ciências ULisboa

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

INTOL

Byzantine Fault-Tolerant Services

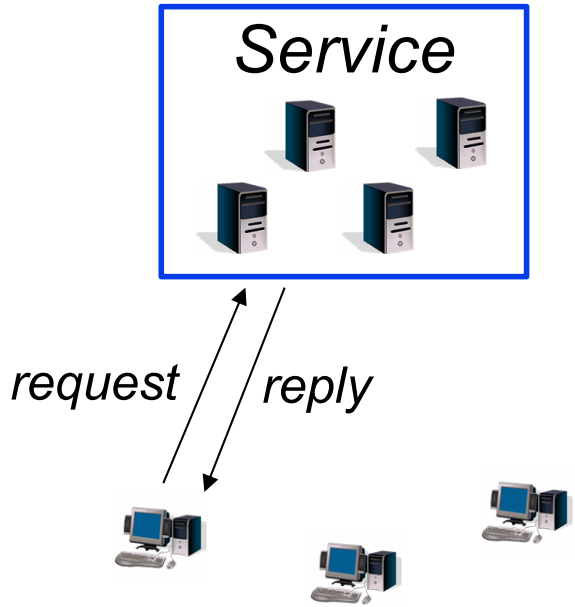
- **Consider a server accessed by several clients**
- **How to make it Byzantine fault-tolerant (BFT)?**
  - **<u>Byzantine faults</u>: a faulty process (client or server) can deviate arbitrarily from its specification, i.e., upon a failure, anything can happen. Can model hardware defects, software bugs and <u>intrusions</u>!**



Ciências ULisboa
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.
INTOL

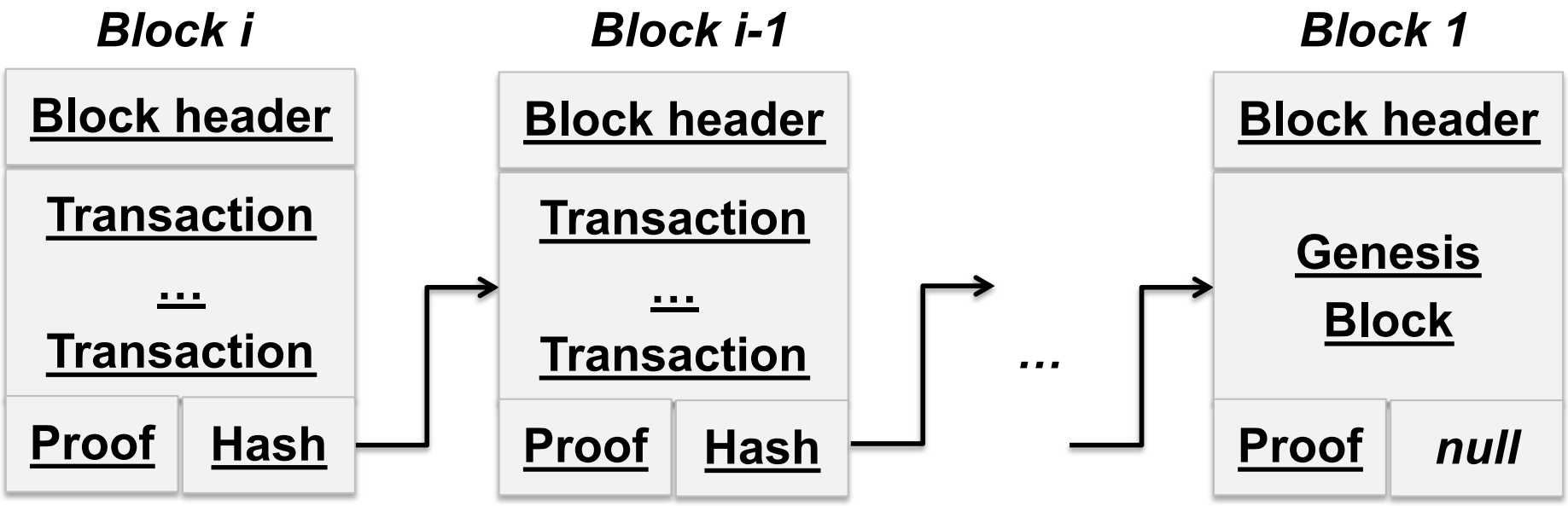
Byzantine Fault-Tolerant Services

- **Which services can benefit from this?**
  - **Typically, a trusted third party that we want to make trustworthy**
  - **Anything that can be modelled as a deterministic state machine**
    - » **More formally:** $(State_{i+1}, reply) = \underline{Service}(State_i, request)$
- **Examples**
  - **Public-Key Infrastructure**
  - **SCADA Master**
  - **Database Management System**
  - **File Repository**
  - **(Zookeeper-like) Coordination Service**
  - **(Stateful) Firewall**
  - **…**
  - **Payment System (or Digital Money)**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

An Example: Blockchains



$Hash_{Block\ i} = H(Block\ i\text{-}1)$

Integrity, auditability, non-repudiation properties, i.e., a malicious party cannot (undetectably):

1. Insert/Modify a block in its copy of the chain
2. Delete a block from the middle of its copy of chain
3. Invalidate or repudiate any of the blocks in a copy of an honest party

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

An Example: Blockchains

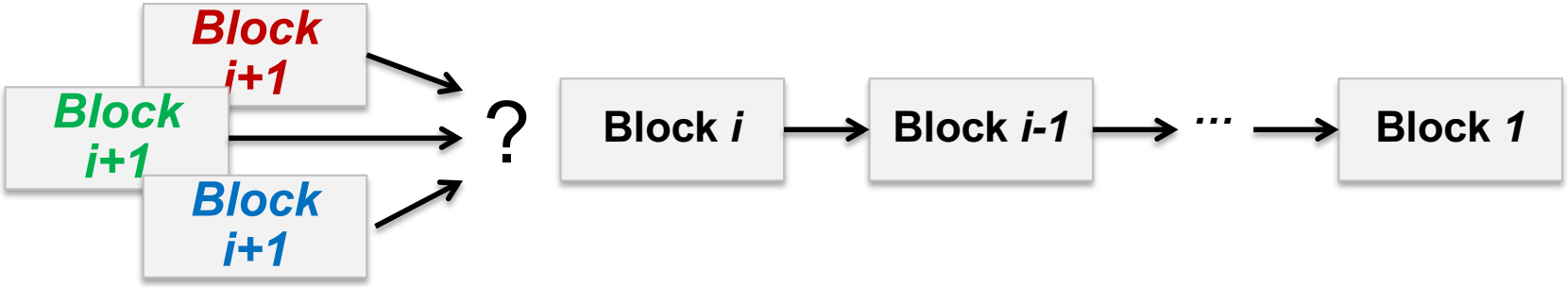
- **Blockchain applications (as of 2023):**
  - **Money**
    - » **Underdeveloped economies, Private cashless society, Stablecoins, etc.**
  - **Decentralized Finance (DeFi)**
    - » **Every financial product can be implemented on blockchains**
  - **Digital Identity**
    - » **Decentralized identities, privacy-preserving id providers, proof-of-personhood**
  - **DAO (Decentralized Autonomous Organizations)**
    - » **On-chain governance of assets and procedures**
  - **Hybrid applications**
    - » **Voting, games, registries, etc.**

Source: https://vitalik.eth.limo/general/2022/12/05/excited.html

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

An Example: Blockchains

- **Is it just a secure log of transactions?**
  - **Yes, but now it is called <u>Secure Ledger</u>**
- **What is new?**
  - **It is distributed, maintained in a P2P way, a <u>Distributed Secure Ledger</u>**
- **It requires solving <u>distributed consensus</u> with Byzantine faults**
  - **Given a blockchain of *i* blocks, and several proposals for block *i+1*, how to decide (in a distributed way) which proposal to adopt?**



Ciências ULisboa
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.
INTOL

Byzantine Consensus

- **The <u>Byzantine distributed consensus</u> problem provides processes an operation *propose()* which takes a value as an input and returns a value.**
  - **If a process p_i invokes *propose(v)* and obtains the value *w*, we say "p_i *proposes v*", and "p_i *decides w*".**
- **Properties of the consensus (one-shot) problem:**
  - **<u>Validity</u>: If all correct processes propose the same value *v*, only *v* can be decided.**
  - **<u>Agreement</u>: No two correct processes decide different values.**
  - **<u>Termination</u>: Each correct process decides a value.**
- **Variants of <u>Validity</u>:**
  - **The decided value is proposed by a correct process.**
  - **(More useful) The decided value satisfy a validity predicate *V()*.**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Distributed system models

- A distributed system is a set of connected process sharing some part of its state
- Two main components:
  - A set of process (usually we will consider a set of *n* servers and an unknown number of clients)
  - These processes communicate through bidirectional links
- Process can also communicate through shared memory or broadcast networks
  - There are a lot of possible system models… we will focus on the ones that represent current distributed systems for the Internet
- The system model must be chosen in such a way to increase the coverage of our system

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Distributed system models

- **Processes:**
  - Timing assumptions
    - » The process have access to real-time clocks?
    - » These clocks are synchronized?
    - » The process computations take a bounded amount of time?
    - » The process computations eventually take a bounded amount of time?
  - Fault model
    - » Omissive (e.g., crash) or assertive (e.g., Byzantine)?
  - Memory
    - » The process has access to stable memory?

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Distributed system models

- **Communication:**
  - **Topology:**
    - » **Everyone is (directly) connected to everyone**
  - **Timing assumptions**
    - » **Communication delay is bounded**
    - » **Communication delay is eventually bounded**
  - **Fault model**
    - » **Channel can lose messages**
    - » **Channel can modify messages**
  - **Authenticated or not**
    - » **The channels are authenticated (i.e., it is impossible to impersonate the sender or receiver of a message?)**
  - **Ordered delivery or not**
    - » **Messages are delivered in FIFO order**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Summary of the Fault-tolerant Distributed Systems Course 1/2

- System models are what define your assumptions about the environment
- They also define what you can and cannot do in your system (it does not matter how smart you are, good engineering cannot beat impossibility results ☺)
- They should be defined carefully to ensure coverage of the assumptions in the real world
- We do not have reliable channels in the real world, only fair channels
- Reliable broadcast can be implemented without time assumptions (asynchronous systems)
- Replication (usually) needs consensus

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Summary of the Fault-tolerant Distributed Systems Course 2/2

- Consensus is equivalent to atomic (or total order) broadcast, which is equivalent to a distributed ledger
- (Crash) Fault-tolerant consensus:
  - Cannot be solved deterministically in asynchronous systems
  - Can be solved probabilistically by using randomization
  - Can be solved in eventually synchronous systems
  - *f+1* (out of *n*) processes are not sufficient to solve consensus in non-synchronous systems. Typically, *2f+1* processes are required.
- Failure detectors allow us to encapsulate the use of time in many distributed computing problems
  - Perfect failure detectors only exist in synchronous systems
  - Unreliable failure detectors are sufficient to solve consensus and they can be implemented on eventually synchronous systems

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Our Failure Model

- **Processes can fail in a Byzantine way:**
  - **Crash, disobey the protocol, send contradictory messages, collude with other malicious processes,...**
- **Communication is reliable and authenticated**

- **Let's start with a synchronous system. Next week we will consider an asynchronous system.**

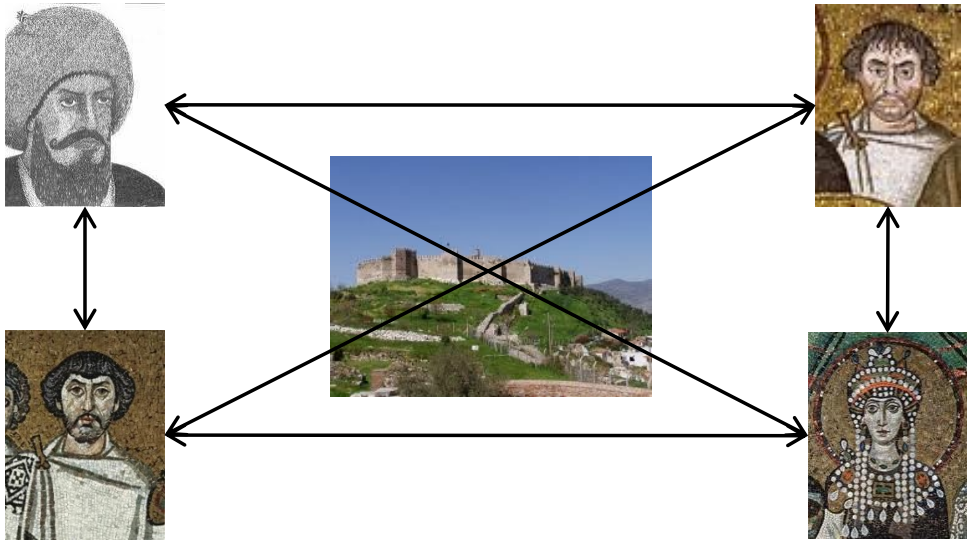
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Fundamentals of Byzantine Fault-tolerant Synchronous Distributed Systems

MSI – MEI – MI – MDS
2021/2022

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine agreement (BA)



- **A classical problem illustrating how hard is to solve consensus**
  - **Several Byzantine generals, in face of an enemy army, must decide whether to attack or to retreat, but they cannot meet, they can only do so *by sending messages to each other***
  - **Most generals are loyal to each other (correct) but some are traitors (faulty)**
  - **In favorable conditions, the combined force of the loyal armies can win**
  - **However, unless all loyal generals attack together, they'll lose**
  - **Traitors will, maliciously, try to prevent agreement from being reached**
  - **Traitors may invent messages, omit some or all messages, send conflicting information, etc.**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

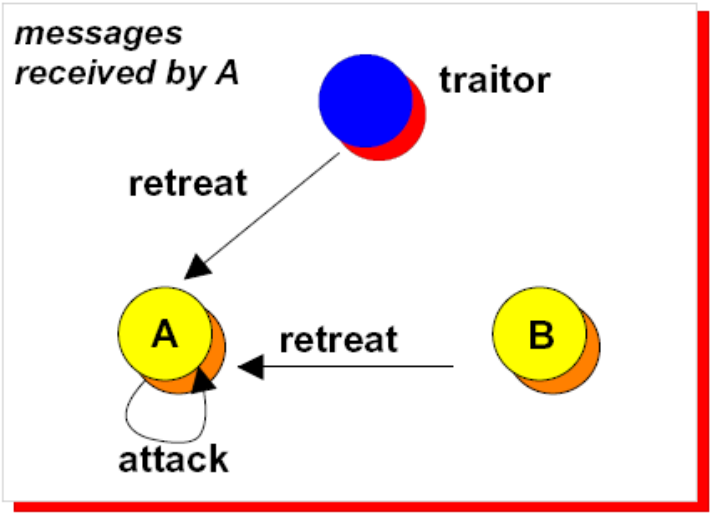
Byzantine agreement (BA)

- The BA protocol operates in synchronous rounds
- In each round, generals send messages to each other
- Loyal generals *must agree on a single binary value* despite the action of traitors
- The initial value proposed by each loyal general consists of his own assessment of the correct decision: to *attack* or *retreat*
- Loyal generals have pre-agreed that they should follow the *majority* and, in case of ties, *retreat*

- ***How many traitors are sufficient to prevent agreement?***

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

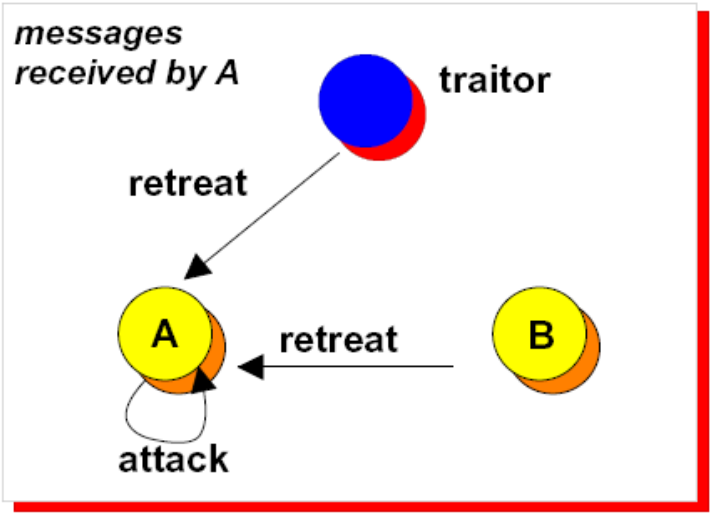
Byzantine agreement

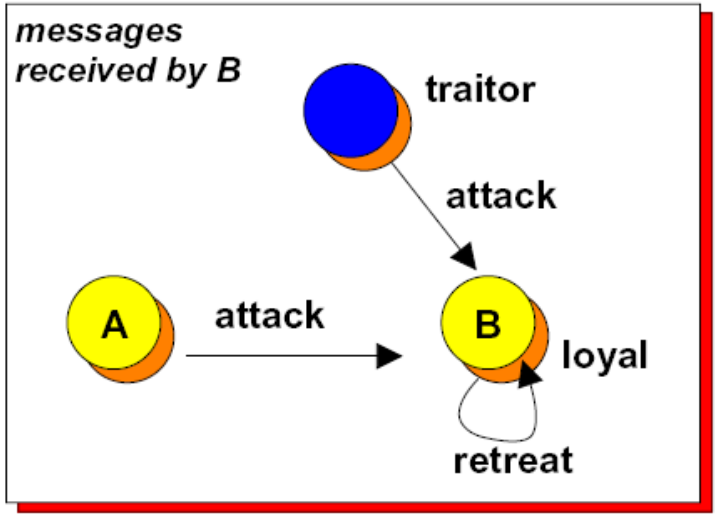


- **Scenario:**
  - Three generals, two of them loyal
  - loyal generals: A, wishes to attack; B, believes they must retreat.
  - A receives two retreat messages, one from B and one from the traitor
- **Intuitively, with majority vote, a correct decision should be possible, right?**
  - If we discarded the traitor's opinion, the decision would be retreat anyway (it's a tie)
  - Unfortunately, *A cannot safely decide…*

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine agreement

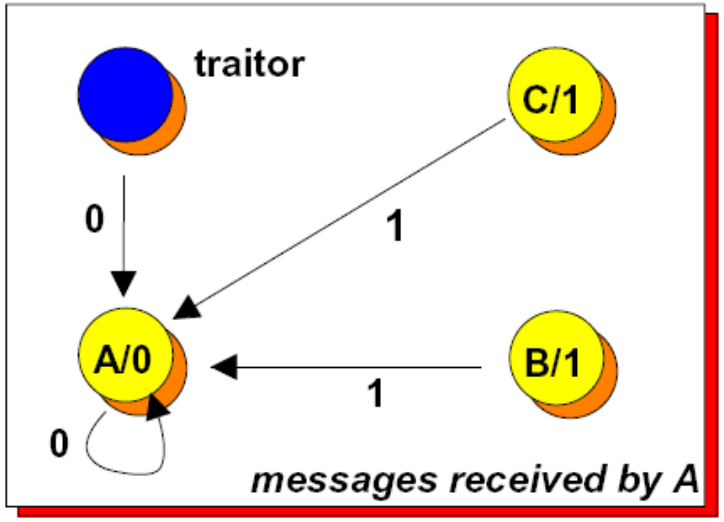




- **Why cannot A safely decide?**
- **The traitor can send a conflicting message to B supporting A's proposal to attack**
- **A simple majority would force *A to retreat and B to attack*!**
  - **For A: (attack, retreat, retretat) → retreat**
  - **For B: (attack, retreat, attack) → attack**
- **This breaks Consensus' Agreement property, so *2f+1 generals are not enough for f traitors***

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

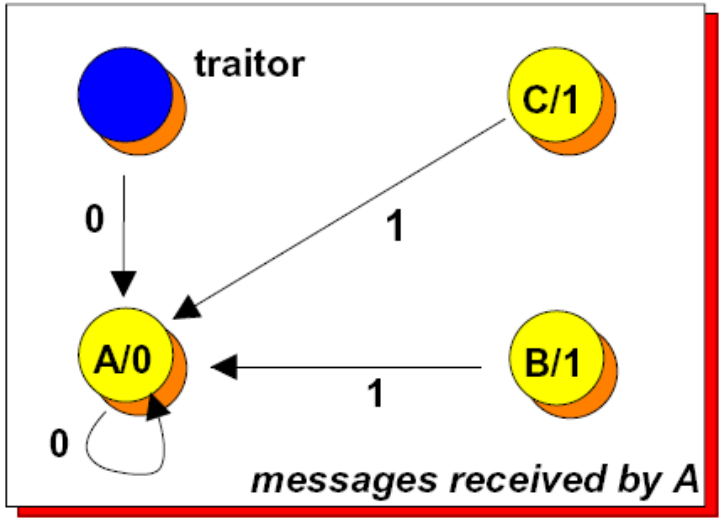
Byzantine agreement

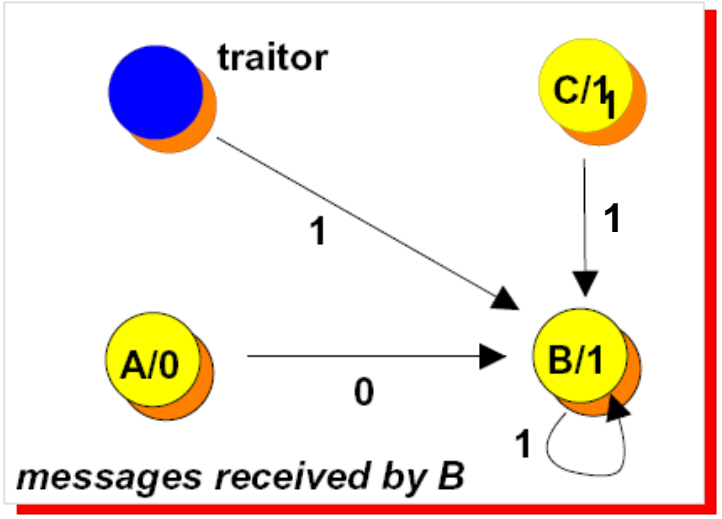


- **Let us add an additional loyal general to the system**
- **Scenario:**
  - **'retreat' is 0 and 'attack' is 1; four generals, three of them loyal**
  - **Loyal generals: B and C, wish to attack; A, believes they must retreat**
- **We have a majority of loyals for attack, so it must work:**
  - **A receives two retreat messages, one from itself and one from the traitor**
  - **That's a tie: decision would be retreat**
  - **Can correct agreement now be reached?**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine agreement





- **No, here's why it still doesn't work:**
  - Normally (traitor off) decision should be attack as B,C propose attack
  - However, A has two retreat and two attack proposals => retreat
  - Worse: B has one retreat and three attack proposals => attack
- **By sending an attack vote to B and a retreat vote to A, the traitor can force A and B to disagree**
- **So, even *3f+1 generals are not enough for f traitors?!***
  - Not in one communication round, but maybe in more rounds…

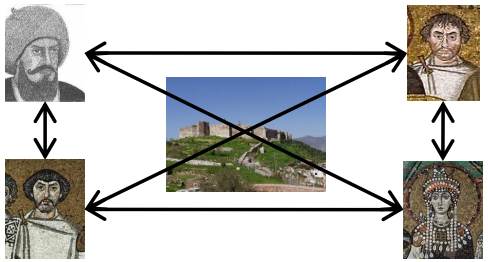
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine agreement

- **Let us add an additional round to the protocol**
  - **Generals will firstly agree about the vote of each general, and only after that they'll decide what to do (based on majority vote)**
  - **For each sender *p*, the other three remaining processes exchange the values they have received from *p* to agree on its value**
  - **This "reliable broadcast" is defined by the Byzantine generals problem**
- **Byzantine generals' problem (formal definition):**

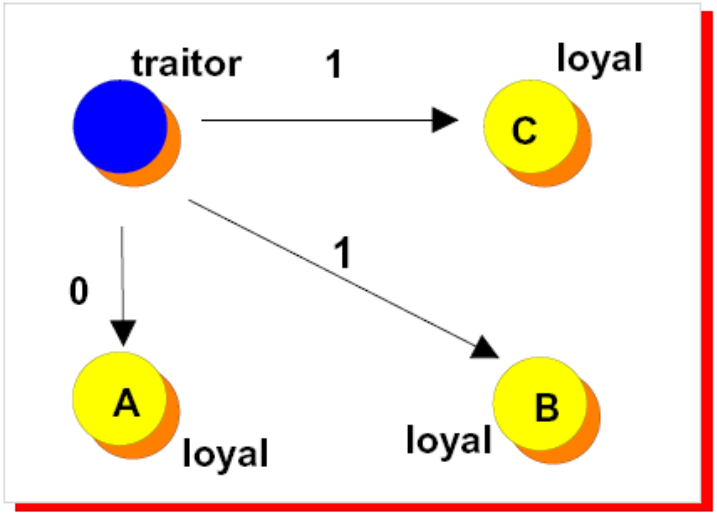
  IC1: All loyal generals obey the same order

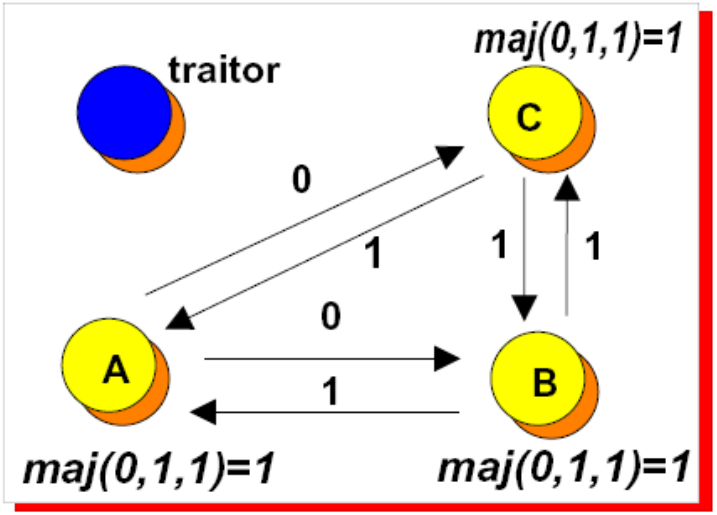
  IC2: If the commanding general is loyal, then every other loyal general obey the order he sends
- **This is also called interactive consistency or *Byzantine (terminating) reliable broadcast***
- **Two situations arise:**
  - **The sender is faulty**
  - **The sender is correct**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine generals problem
(faulty sender)

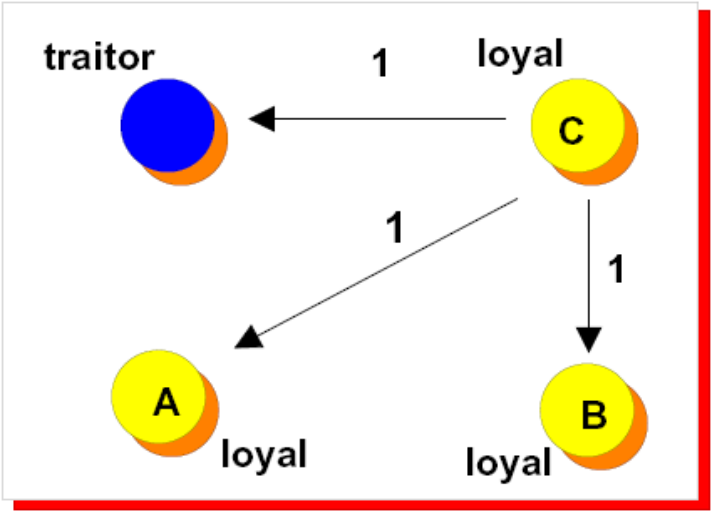


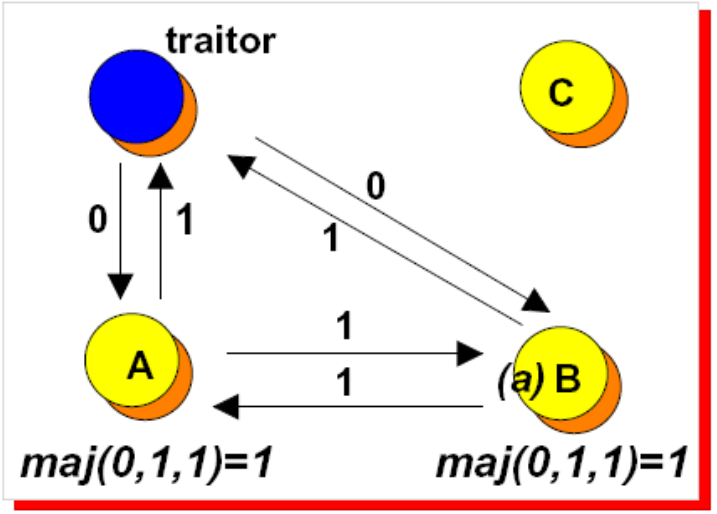


- **In this case the traitor can send inconsistent votes to the loyal processes *(0,1,1)***
- **Since all remaining generals are loyal, after exchanging among them the value received from the traitor, they have the same set of values**
- **If they use a majority criteria, all correct processes will select the same final value, i.e., *majority(0,1,1)=1***

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine generals problem
(sender correct)





- **In this case, C disseminates the same value (1) to all processes**
- **Remaining three processes exchange the value received**
  - **two correct processes (A, B) forward the value originally sent by C**
  - **faulty process forwards some arbitrary value *x***
  - **since there is a majority of correct values, the value originally sent by C will be chosen: *majority(x,1,1)=1***
  - **in this case, *majority(0,1,1)=1***

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine generals problem
(Oral Messages algorithm)

Algorithm OM(0):

(1) Commander sends its order to other processes.

(2) Each process decides the received value or 'retreat' if no value was received.

Algorithm OM(f), *f > 0 (f is the maximum number of faulty processes)*:

(1) Commander send its order to other processes.

(2) Let v_i be the value received by processes *i* from the commander, or 'retreat' if no value was received. Process *i* disseminate v_i (as a commander) to the other *n-2* non-commander processes using OM(*f –1*).

(3) For each *i* and $j \neq i$, let v_j be the value *j* received from *i* in step (2). The value decided by *i* is $majority(v_1, v_2, \ldots, v_{n-1})$.

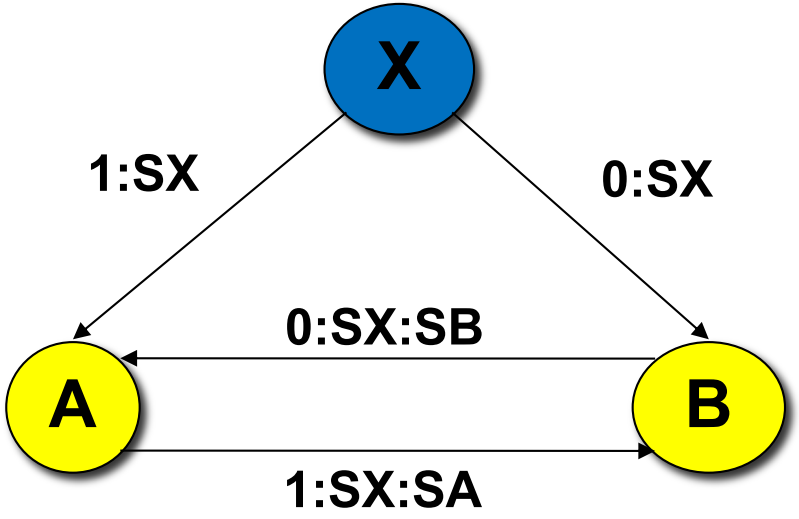
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine generals problem
(with authentication)

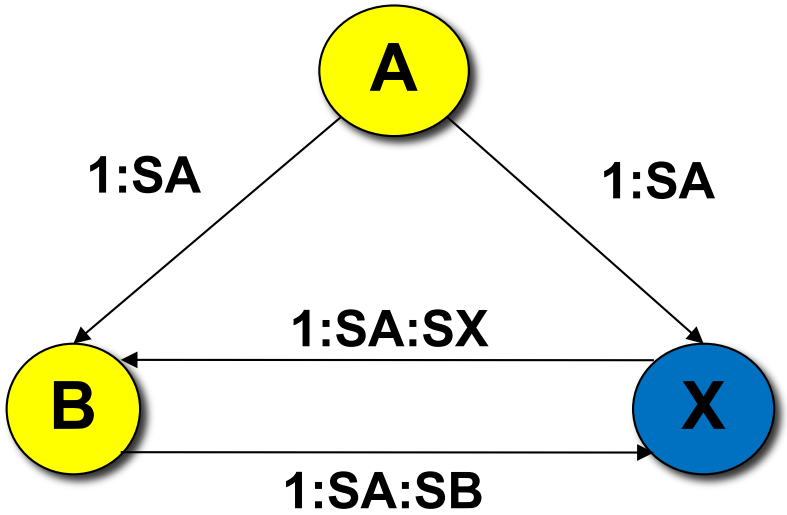
- **The power of digital signatures:**
  - Let's consider what happens if the messages exchanged on the previous algorithm are signed
  - Important: Digital signatures are stronger than authenticated channels (or MACs) as they can be verified by a third party
- **Assumptions about digital signatures**
  - A loyal general's signature cannot be forged, and any alteration of the contents of his signed messages can be detected
  - Anyone can verify the authenticity of a general's signature. This usually requires a PKI (Public Key Infrastructure) or that the keys were distributed in a trusted setup phase.
- **Messages with invalid signatures are discarded and their senders are considered faulty**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Byzantine generals problem
(with authentication)



traitor sender



traitor receiver

- **Messages are now signed… and relayed values are also signed by relay nodes**
- **If the traitor sends contradictory values, he will be discovered**
- **With digital signatures, we need just $n \geq f+2$ processes to solve the Byzantine generals problem**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Bibliography

- **Intro to blockchain from next lectures**
- *https://vitalik.eth.limo/general/2022/12/05/excited.html*
- **"Distributed Fault Tolerance" course slides**
- **L. Lamport, R. Shostak, M. Pease. *The Byzantine Generals Problem*. ACM Trans. On Programming Languages and Systems. Vol 4, Num 3. 1982.**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.